INSTITUTO POLITÉCNICO NACIONAL
UNIDAD PROFESIONAL
INTERDISCIPLINARIA EN
INGENIERÍA Y TECNOLOGÍAS
AVANZADAS

MULTIMEDIA

PRÁCTICA 6
“Esteganografía Avanzada”

Alumno: De la Rosa Valdes Itztli Fernanda

Boleta: 2024640062

Derivación de clave y cifrado XOR: Implemente las funciones de cifrado y descifrado del mensaje:

```python
def derivar_clave(password: str, longitud: int) -> bytes:
    '''Genera una clave de longitud arbitraria usando SHA-256 [cite: 147-156]'''
    clave = b''
    contador = 0
    while len(clave) < longitud:
        bloque = hashlib.sha256(password.encode() + struct.pack('<I', contador)).digest()
        clave += bloque
        contador += 1
    return clave[:longitud]

def cifrar_xor(mensaje: bytes, password: str) -> bytes:
    '''Aplica XOR entre el mensaje y la clave derivada [cite: 157-158]'''
    clave = derivar_clave(password, len(mensaje))
    return bytes(m ^ k for m, k in zip(mensaje, clave))
```

Selección pseudoaleatoria de posiciones portadoras: Implemente la función que genere la lista de índices de bytes portadores:

```python
def seleccionar_posiciones(total_bytes_imagen: int, n_bits: int, password: str) -> list:
    '''Selecciona índices únicos basados en una semilla de la contraseña [cite: 161-168]'''
    # Convertimos el hash de la contraseña en una semilla numérica [cite: 167-168]
    seed = int.from_bytes(hashlib.sha256(password.encode()).digest()[:8], 'big')
    rng = random.Random(seed)
    # Seleccionamos n_bits posiciones sin repetir
    posiciones = rng.sample(range(total_bytes_imagen), n_bits)
    return posiciones
```

Función de incrustación segura: Implemente embed_secure() que integre los pasos anteriores:

```python
def embed_secure(src_path, dst_path, mensaje, password):
    header, pixels, width, height, row_size = leer_bmp(src_path)
    msg_bytes = mensaje.encode('utf-8')

    # Ciframos el mensaje antes de ocultarlo [cite: 174]
    msg_cifrado = cifrar_xor(msg_bytes, password)

    # Construimos el flujo: Longitud (4 bytes) + Mensaje Cifrado [cite: 175-177]
    datos = struct.pack('>I', len(msg_bytes)) + msg_cifrado

    bits = []
    for byte in datos:
        for i in range(7, -1, -1):
            bits.append((byte >> i) & 1)

    n_bits = len(bits)
    if n_bits > len(pixels):
        raise ValueError('Mensaje demasiado grande para esta imagen [cite: 189]')

    # Seleccionamos las posiciones aleatorias [cite: 195]
    posiciones = seleccionar_posiciones(len(pixels), n_bits, password)

    pixels_mod = bytearray(pixels)
    for pos, bit in zip(posiciones, bits):
        pixels_mod[pos] = (pixels_mod[pos] & 0xFE) | bit # [cite: 198]

    guardar_bmp(dst_path, header, pixels_mod)
    print(f' [OK] {len(msg_bytes)} bytes cifrados e incrustados en {dst_path} [cite: 200]')
```

Función de extracción segura: Implemente extract_secure() para recuperar el mensaje con la contraseña correcta:

```python
def extract_secure(stego_path, password):
    _, pixels, _, _, _ = leer_bmp(stego_path)

    # 1. Recuperar posiciones para los 32 bits de longitud [cite: 205-206]
    pos_longitud = seleccionar_posiciones(len(pixels), 32, password)
    len_bits = [pixels[p] & 1 for p in pos_longitud]

    msg_len = 0
    for b in len_bits:
        msg_len = (msg_len << 1) | b # [cite: 210]

    # 2. Recuperar todas las posiciones (longitud + mensaje) [cite: 212-215]
    total_bits = 32 + msg_len * 8
    todas_pos = seleccionar_posiciones(len(pixels), total_bits, password)

    # Solo nos interesan los bits después de los primeros 32 [cite: 216]
    msg_bits = [pixels[p] & 1 for p in todas_pos[32:]]

    cifrado = bytearray()
    for i in range(0, len(msg_bits), 8):
        byte = 0
        for bit in msg_bits[i:i+8]:
            byte = (byte << 1) | bit
        cifrado.append(byte)

    # 3. Descifrar con la contraseña [cite: 224-225]
    return cifrar_xor(cifrado, password).decode('utf-8')
```

Prueba con contraseña correcta e incorrecta: Verifique el sistema con ambos escenarios y documente los resultados:

```python
# --- PRUEBA PRÁCTICA 2 ---
CLAVE = 'Telemática@2026'
MENSAJE = 'Datos confidenciales de la red 10.0.1.0/24'

print("--- Iniciando Esteganografía Avanzada ---")
embed_secure('imagen.bmp', 'stego_seguro.bmp', MENSAJE, CLAVE)
resultado = extract_secure('stego_seguro.bmp', CLAVE)
print(f'🔒 Mensaje con clave correcta: "{resultado}"')

# Intento con clave errónea
try:
    error = extract_secure('stego_seguro.bmp', 'clave_falsa')
    print(f'🚫 Intento fallido (basura): {error[:20]}...')
except:
    print("🚫 Intento fallido: La clave incorrecta genera datos ilegibles.")
```

```
--- Iniciando Esteganografía Avanzada ---
✅ [OK] 42 bytes cifrados e incrustados en stego_seguro.bmp [cite: 200]
🔒 Mensaje con clave correcta: "Datos confidenciales de la red 10.0.1.0/24"
🚫 Intento fallido: La clave incorrecta genera datos ilegibles.
```

Análisis Estadístico (Chi-cuadrado)

```python
def chi_cuadrado_lsb(filepath):
    '''Calcula el estadístico Chi-cuadrado de los LSBs [cite: 248-261]'''
    _, pixels, _, _, _ = leer_bmp(filepath)

    # Contar cuántos LSB son 0 y cuántos son 1 [cite: 250-254]
    ceros = sum(1 for b in pixels if (b & 1) == 0)
    unos = len(pixels) - ceros

    # El valor esperado es una distribución 50/50 [cite: 255-257]
    esperado = len(pixels) / 2
    chi2 = ((ceros - esperado)**2 + (unos - esperado)**2) / esperado

    print(f'Archivo: {filepath}')
    print(f'  LSBs=0: {ceros} | LSBs=1: {unos} | χ²: {chi2:.4f}')
    if chi2 < 1.0:
        print('  Resultado: Distribución uniforme (Difícil de detectar) [cite: 259]')
    else:
        print('  Resultado: Distribución no uniforme (Sospecha de LSB secuencial)')
    return chi2

# Ejecutar comparativa [cite: 262-265]
print("=== COMPARATIVA ESTADÍSTICA ===")
chi_original = chi_cuadrado_lsb('imagen.bmp')
chi_secuencial = chi_cuadrado_lsb('stego.bmp') # De la Práctica 1
chi_aleatorio = chi_cuadrado_lsb('stego_seguro.bmp') # De la Práctica 2
```

```
=== COMPARATIVA ESTADÍSTICA ===
Archivo: imagen.bmp
  LSBs=0: 393216 | LSBs=1: 393216 | χ²: 0.0000
  Resultado: Distribución uniforme (Difícil de detectar) [cite: 259]
Archivo: stego.bmp
  LSBs=0: 393263 | LSBs=1: 393169 | χ²: 0.0112
  Resultado: Distribución uniforme (Difícil de detectar) [cite: 259]
Archivo: stego_seguro.bmp
  LSBs=0: 393232 | LSBs=1: 393200 | χ²: 0.0013
  Resultado: Distribución uniforme (Difícil de detectar) [cite: 259]
```